Infrastructure as Code with Terraform

Terraform

Table of Contents

Activity 1: Install Terraform on AWS

1.1 Prerequisite

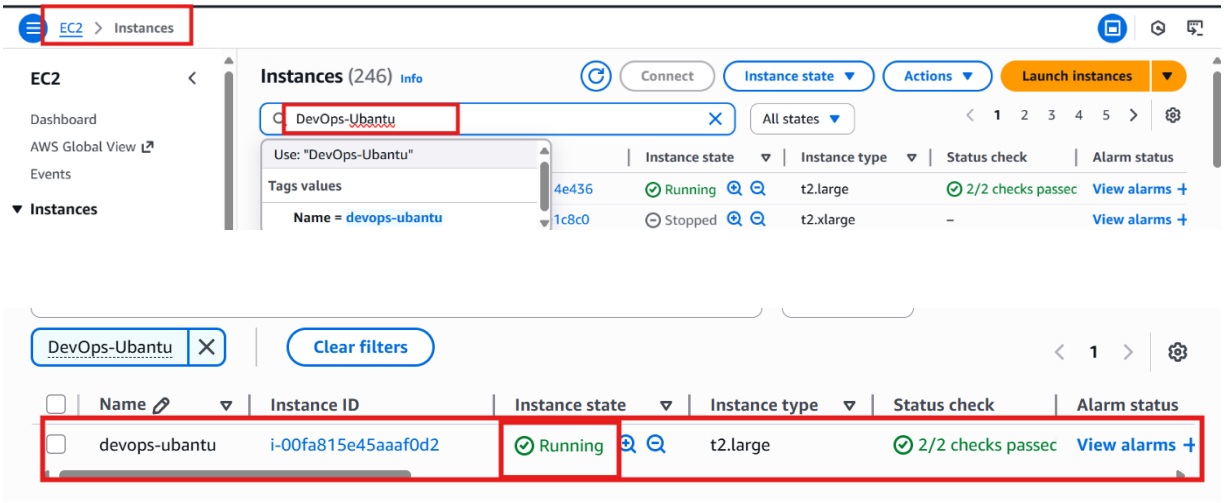
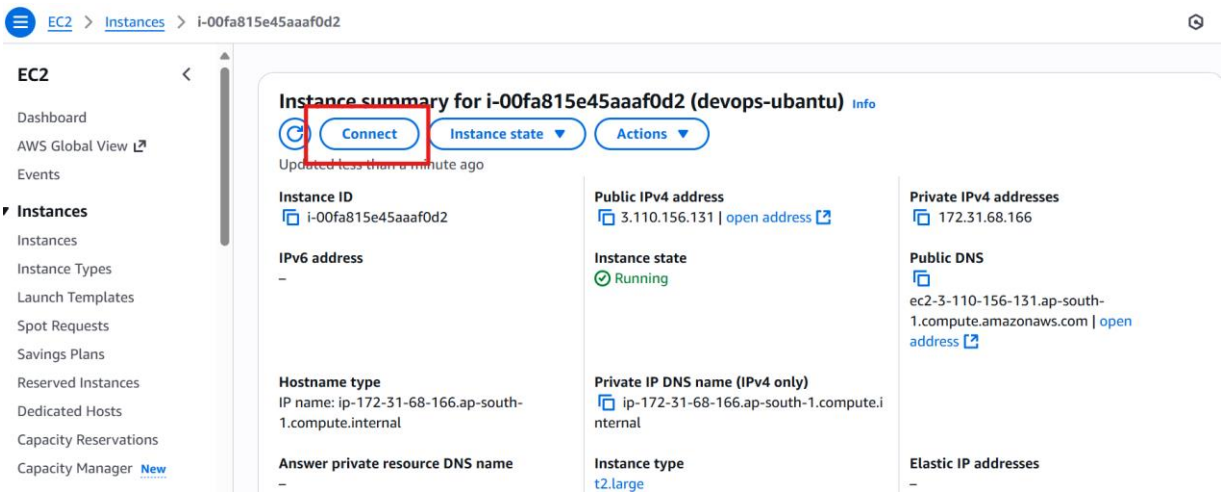
AWS account is ready to use.

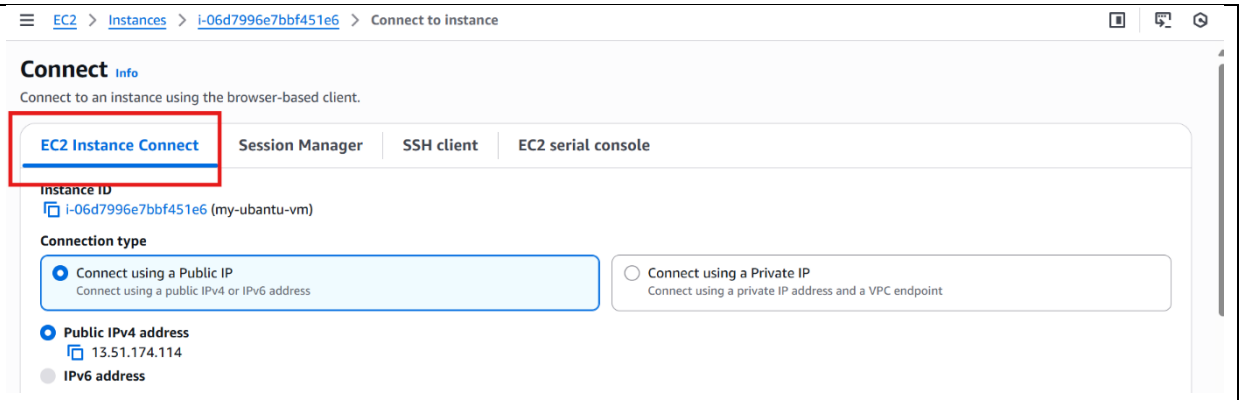
AWS EC2 instance is ready to use.

1.2 Scenario

Launch Shell Editor on AWS to install Terraform.

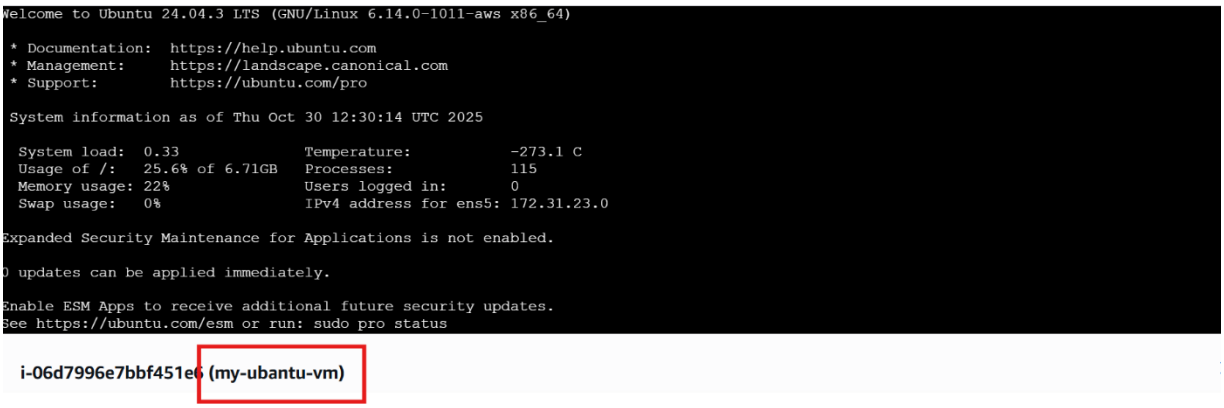
1.3 Steps

1	Open AWS cloud console-> EC2-> instances-> Search instance by name or id  Click the **Connect** button to view the different options for connecting to your AWS instance. 

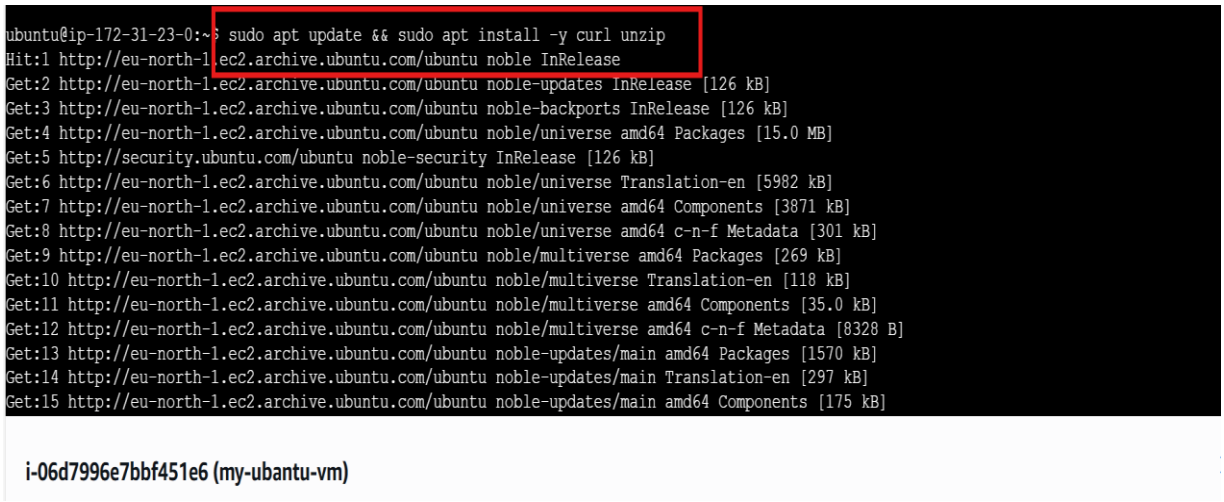


Note: If you are using the Ubuntu Desktop (GUI) version, open a Terminal window and execute the commands listed below.

2 Open EC2 Instance Connect

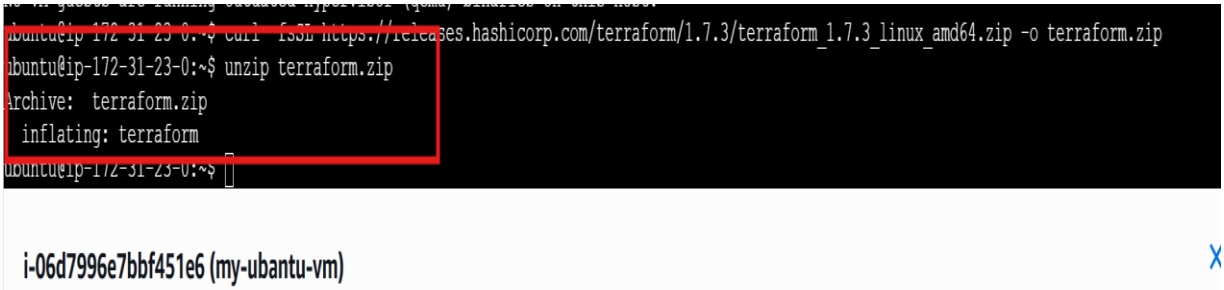
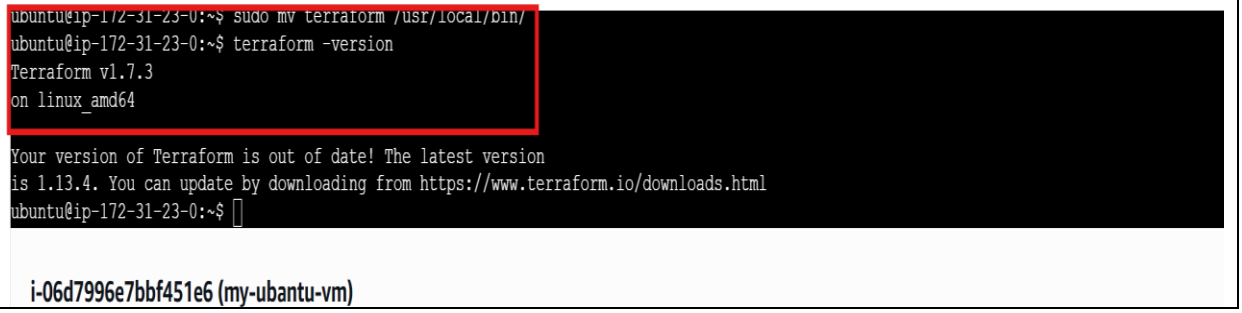


3 Update system
Command: sudo apt update && sudo apt install -y curl unzip



4 Download Terraform

Command:
curl -fsSL https://releases.hashicorp.com/terraform/1.7.3/terraform_1.7.3_linux_amd64.zip -o terraform.zip

	
5	Unzip terraform Command: unzip terraform.zip 
6	Move terraform to the system path Command: sudo mv terraform /usr/local/bin/ 
7	verify terraform installation command: terraform -version 

Activity 2: Authenticate & Configure Terraform with AWS using Terminal

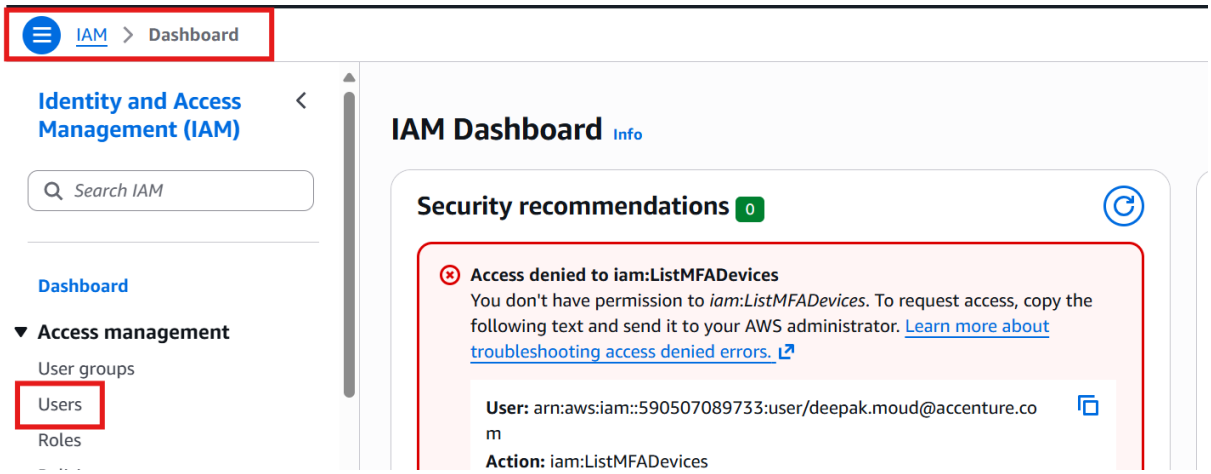
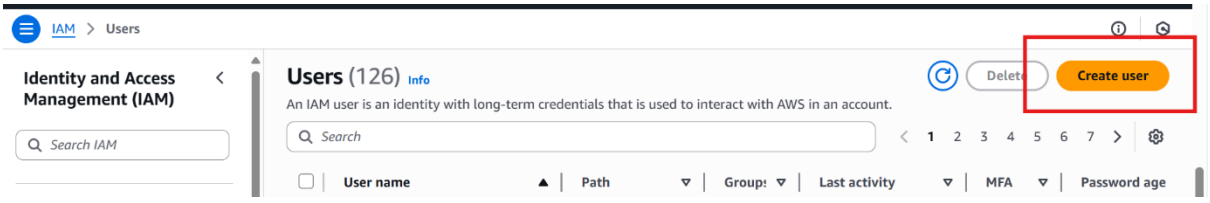
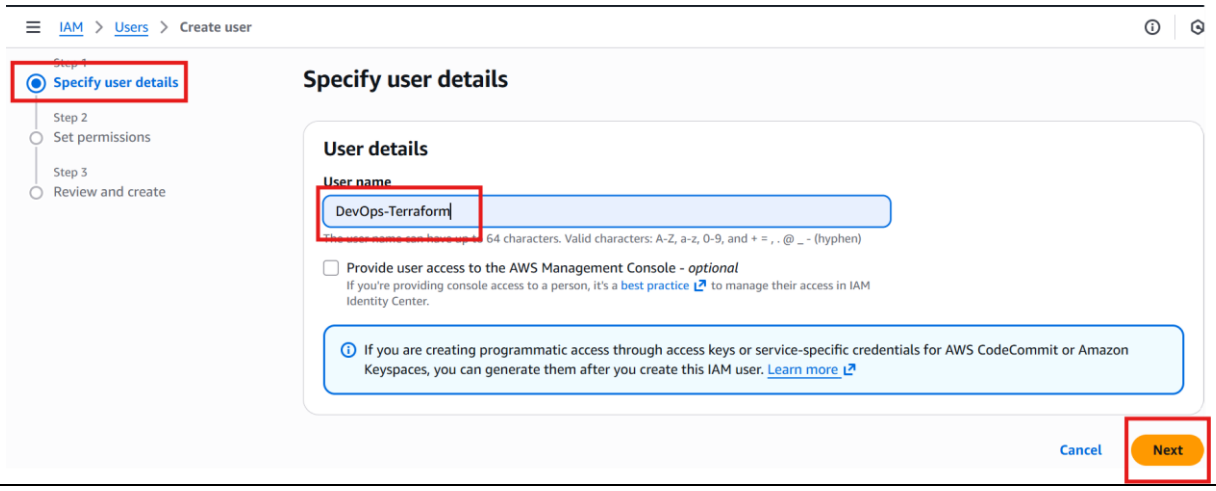
2.1 Prerequisite

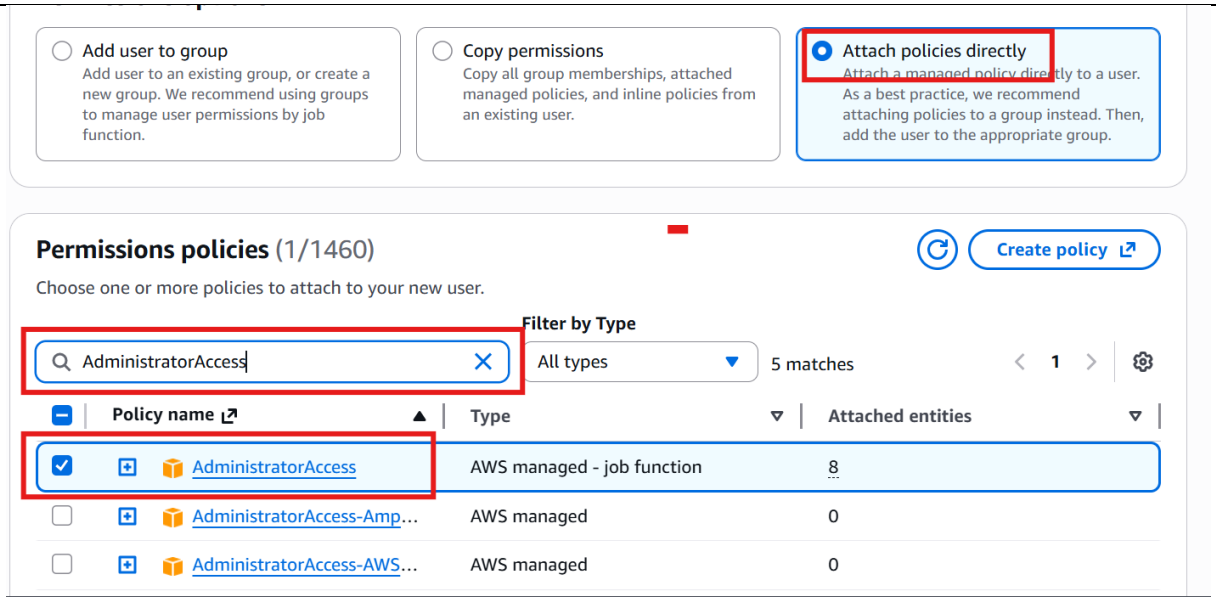
Terraform is installed on AWS instance.

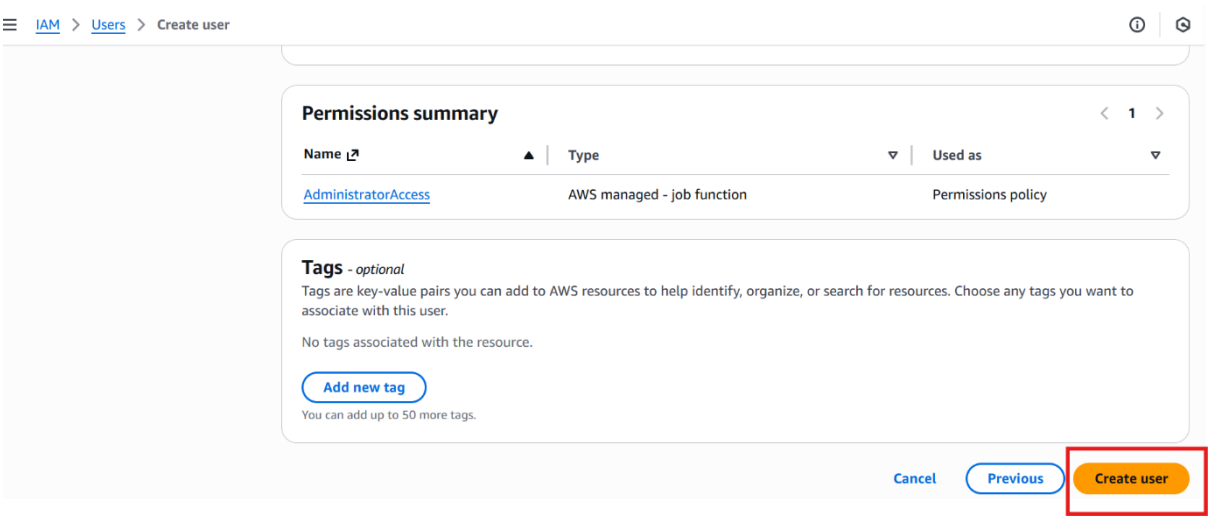
2.2 *Scenario*

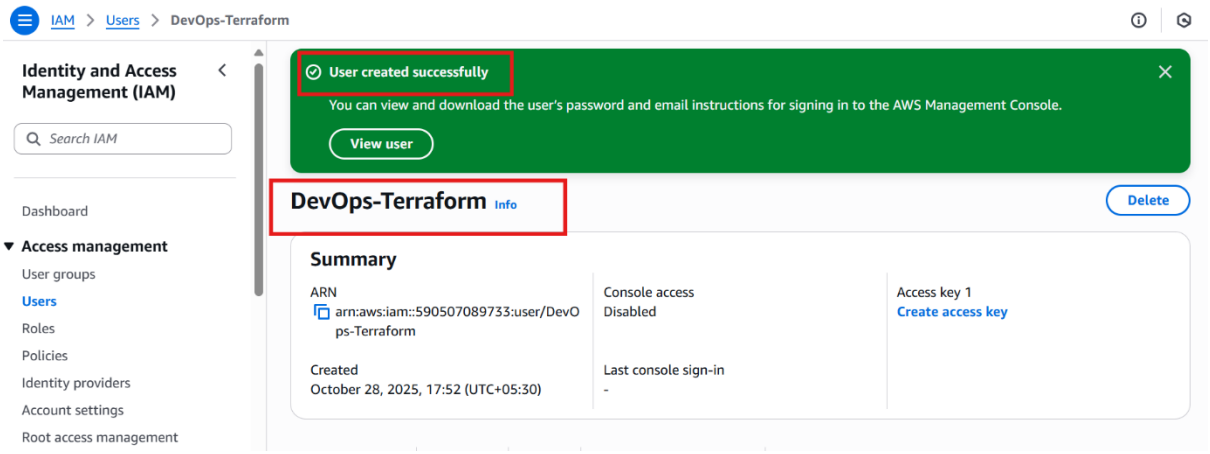
Authenticate & Configure terraform with AWS to execute terraform script to create resources.

2.3 *Steps*

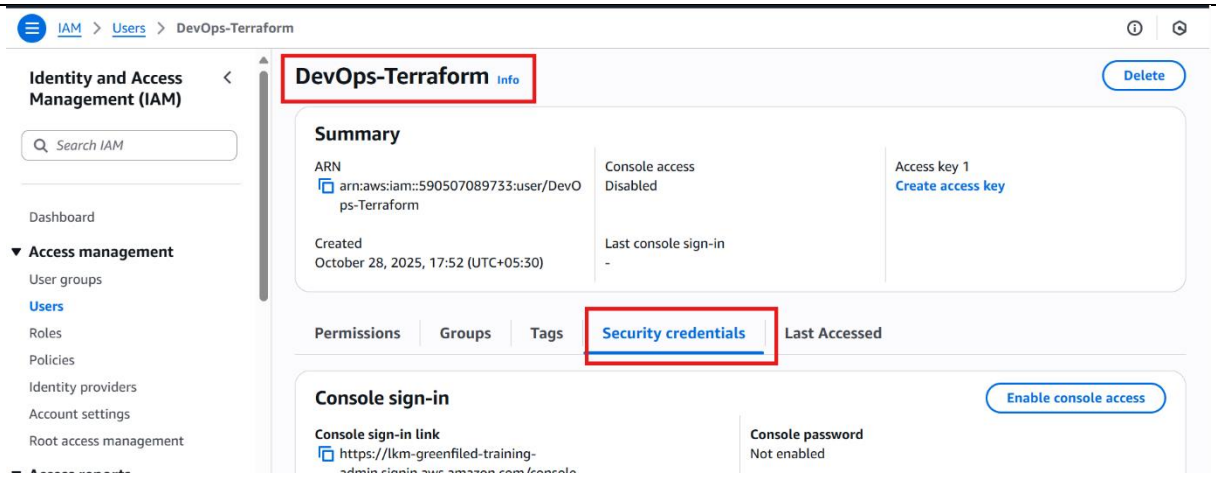
1	Create a service account to authenticate terraform Go to AWS Console-> Click IAM -> Users 
2	Click on Create User 
3	Enter name-> click Next-> Under **Permissions** --> Attach policies directly ---> Add this policy: **AdministratorAccess -> Click next-> Click on Create User.** 



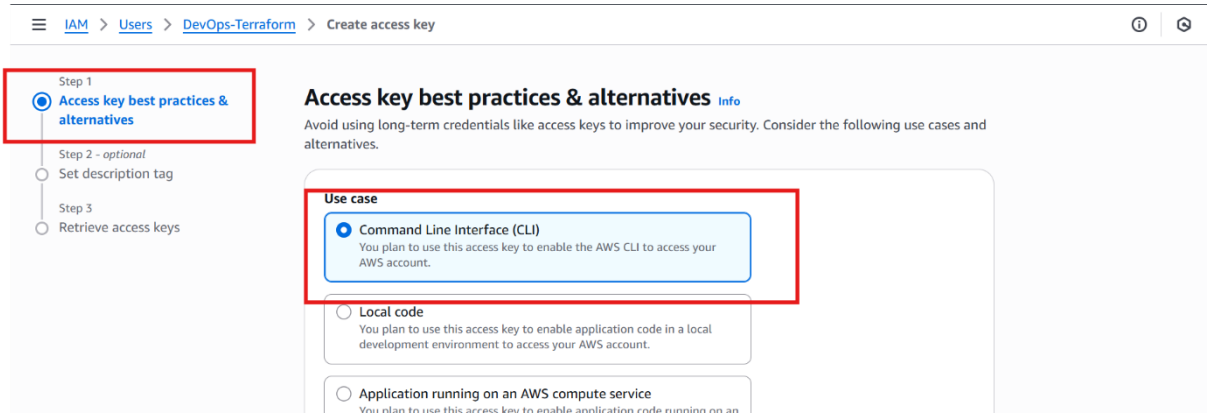


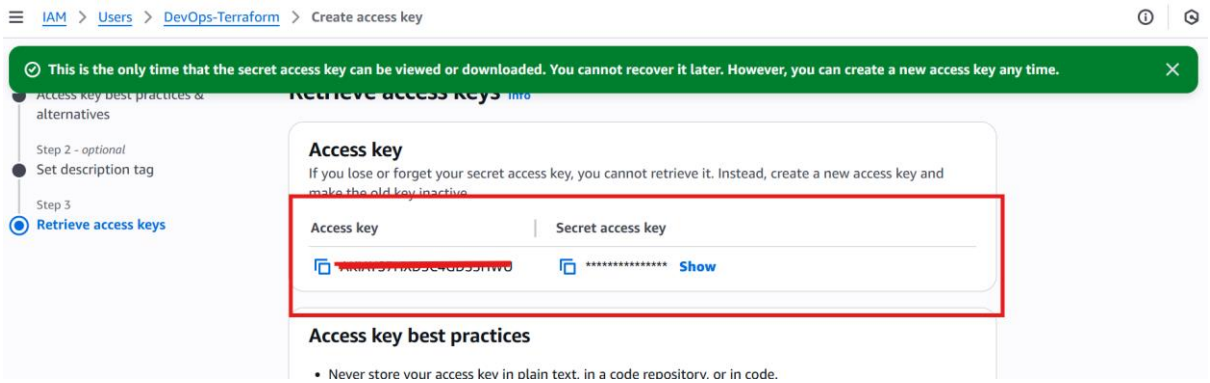


4	Generate access key. Click on New user (DevOps-Terraform)-> **Security credentials** tab



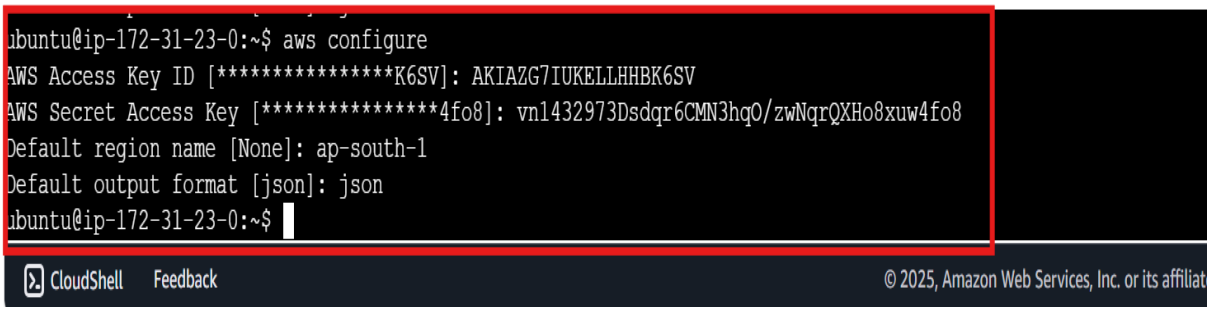
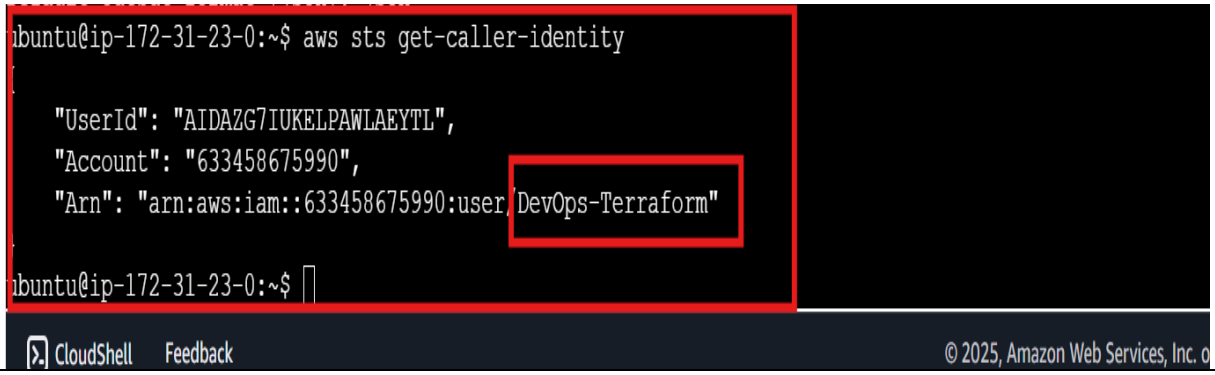
Scroll to **Access keys -> Click Create access key-> Command Line Interface (CLI) → Next → Create**



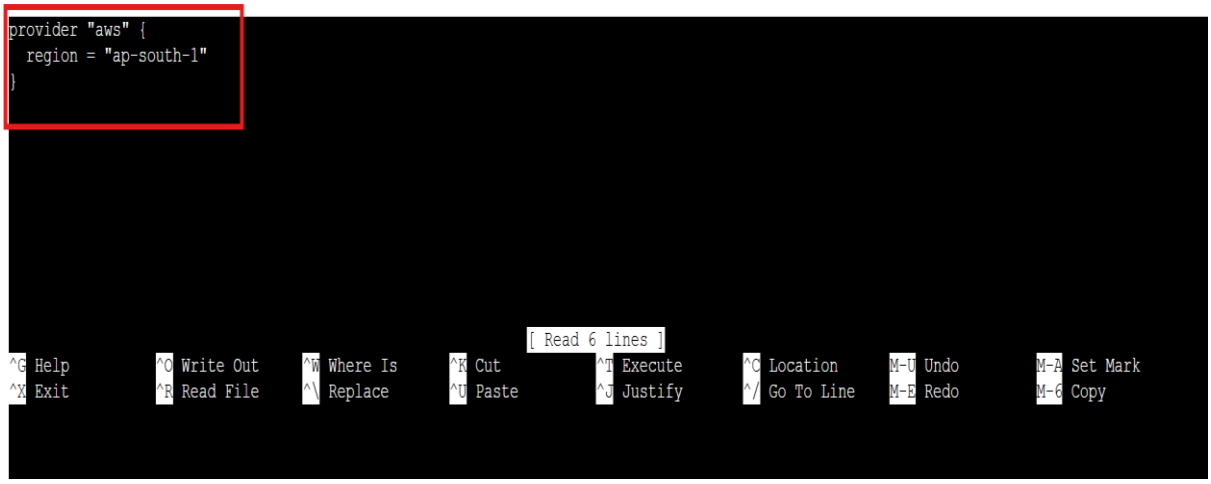
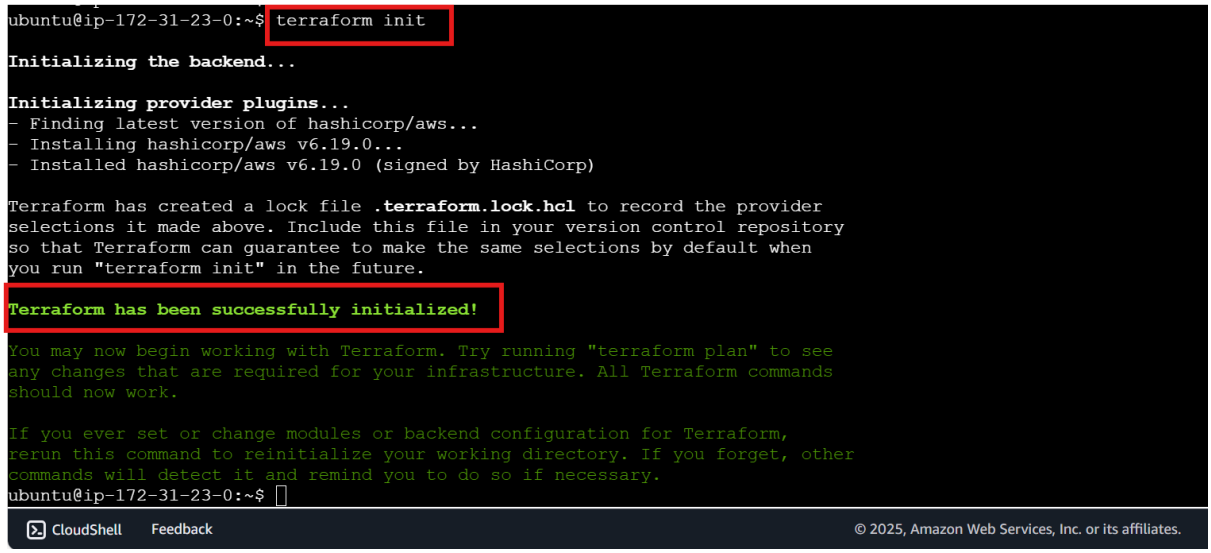
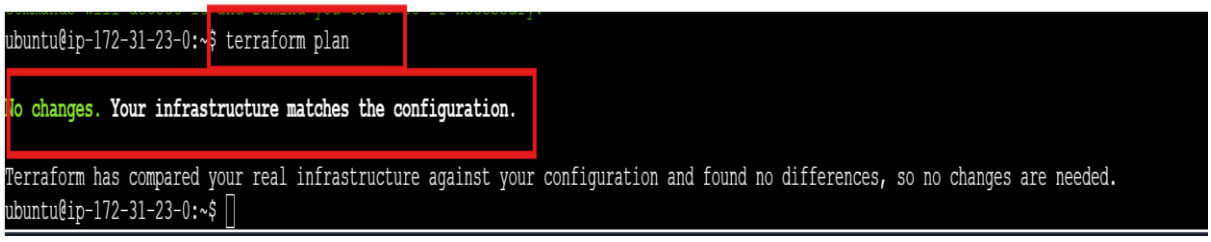


Copy and download keys.

5	Now, configure AWS CLI on Ubantu: Open the terminal and type: "aws configure" When prompted, enter the following details one by one: • **AWS Access Key ID**

	• **AWS Secret Access Key** • **Default Region Name** • **Output Format**  The credentials will be saved in the following locations: **~/.aws/credentials** **~/.aws/config**
6	Now, Verify Configuration: In the terminal, run the following command: **aws sts get-caller-identity** If the configuration is successful, you'll see your AWS account and user details displayed: 
7	Create Terraform configuration file with name "main.tf" Command: nano main.tf  That will open editor.

8	Use Terraform with AWS Authentication
	Paste below terraform script in main.tf through editor
	
	provider "aws" {
	region = "ap-south-1"
	}
	Replace region in above script.

	Press **CTRL + X** → Type **Y** → Press **Enter to save file.** 
9	Run Terraform commands: **terraform init** -> Initialize terraform **terraform plan** -> Check Terraform execution plan  
14	Now Terraform is authenticated with AWS to create resources.

Activity 3: Create AWS EC2 Instance Resource using Terraform

3.1 Prerequisite

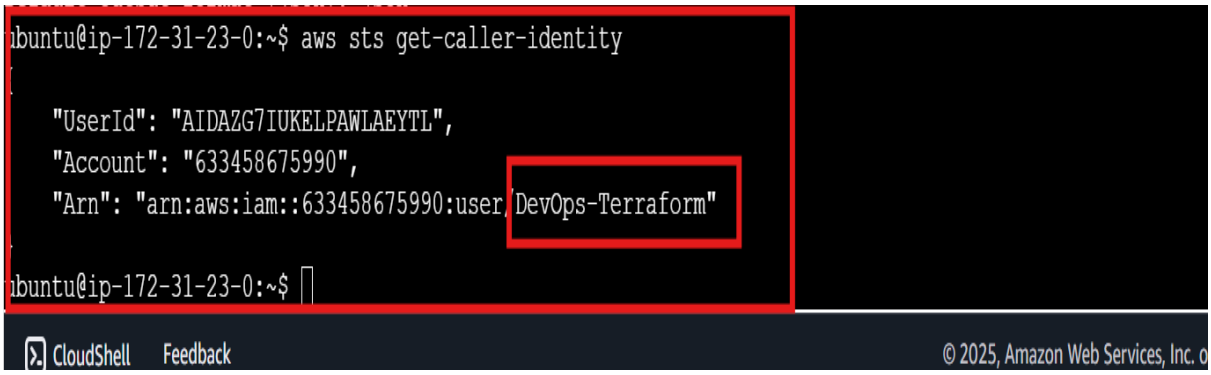
AWS cloud account is ready.

Terraform is authenticated and configured to create resources

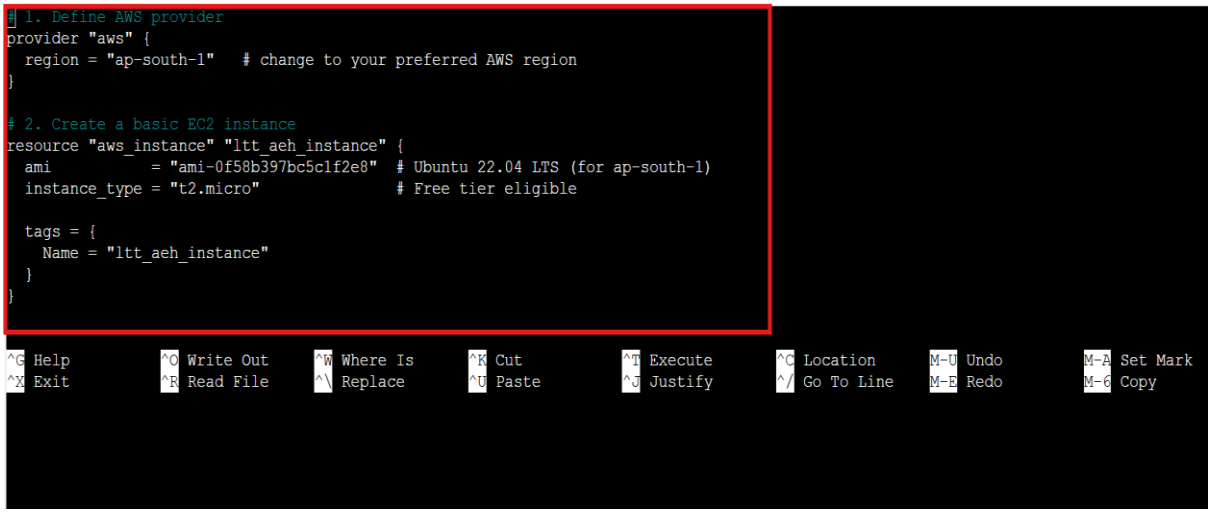
3.2 Scenario

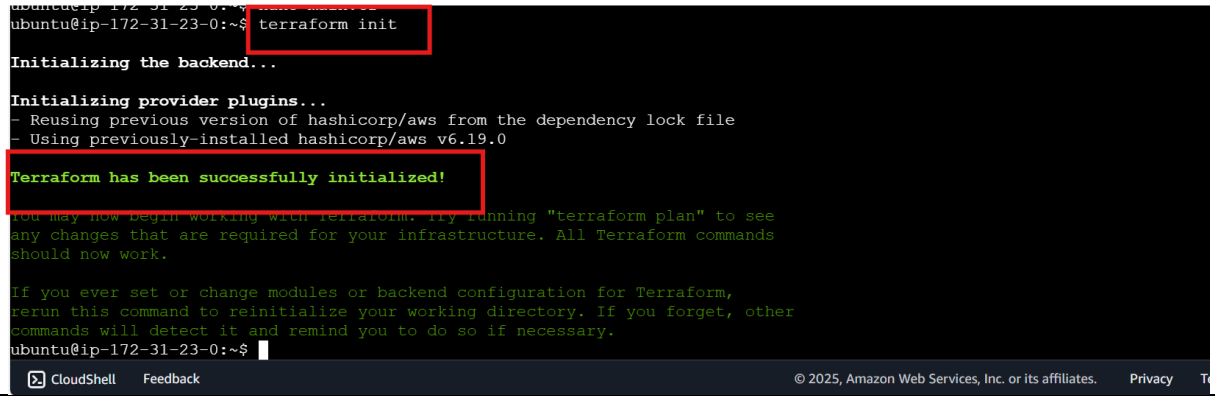
Create EC2 instance using terraform script.

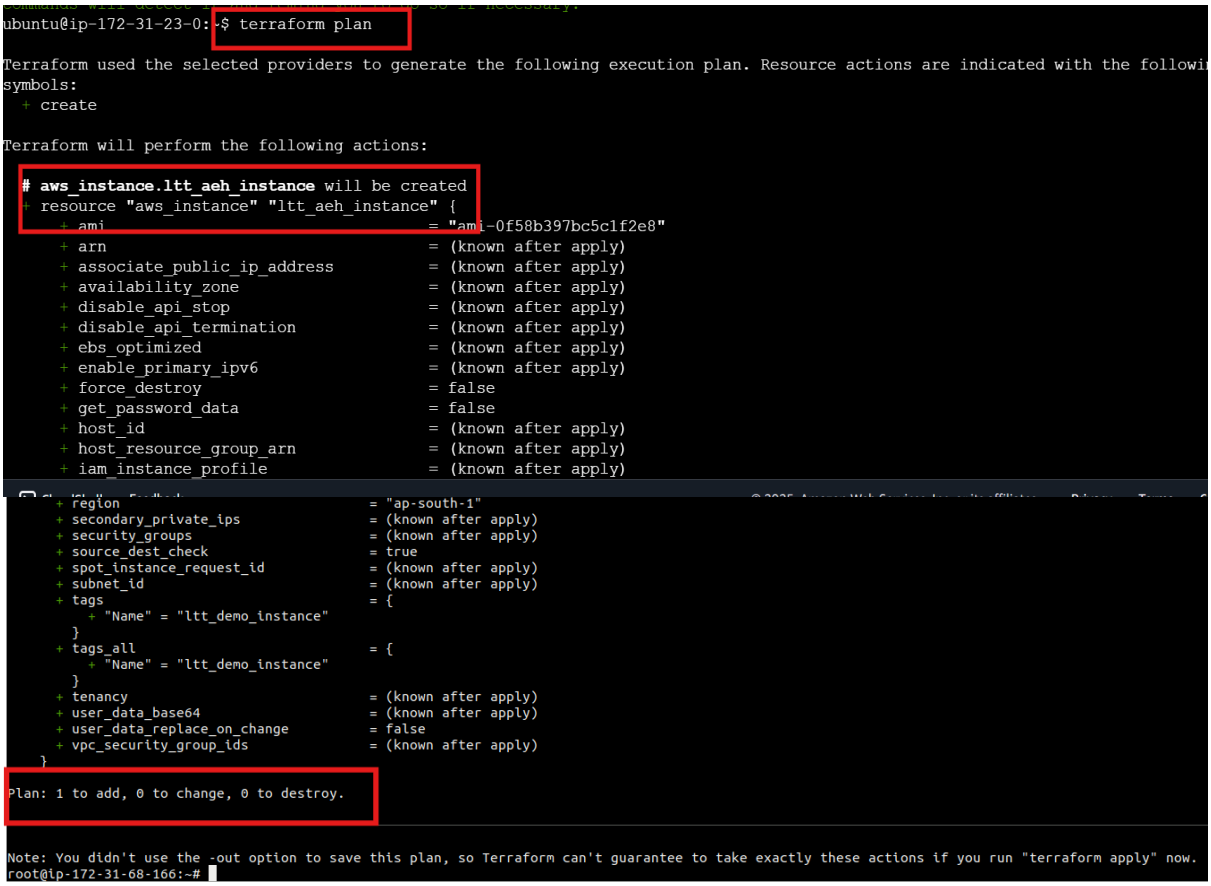
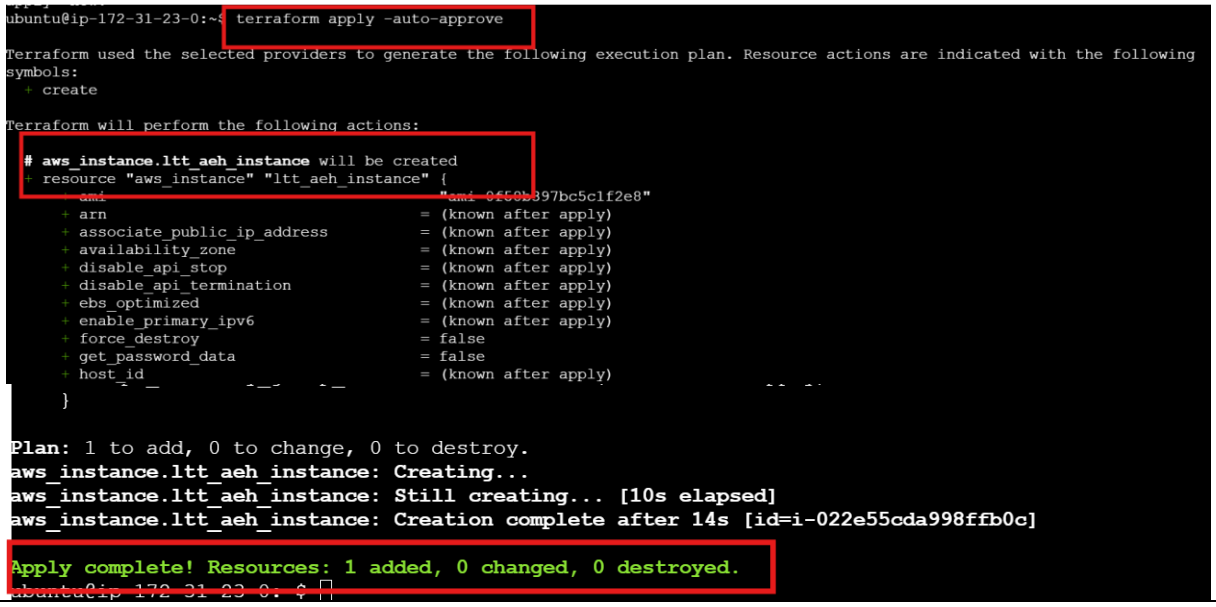
3.3 Steps

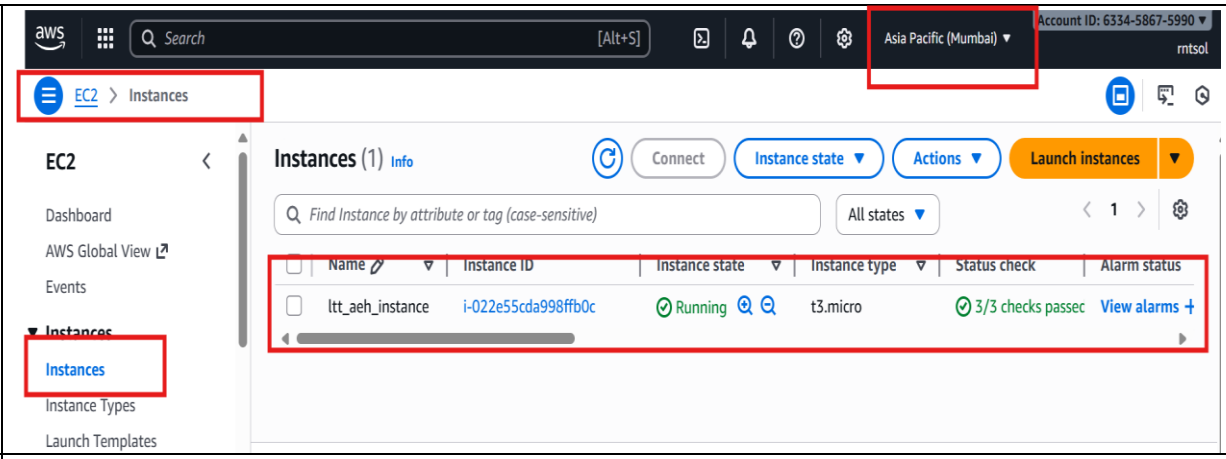
1	Make sure you have authenticated AWS. Verify using : aws sts get-caller-identity  If the configuration is successful, you'll see your AWS account and user details displayed.
2	If AWS authentication is not yet configured, run the "aws configure" command and provide the required credentials when prompted.

4	create main.tf script file to create resources.
	
	execute "nano main.tf"
	paste attached terraform script in this file to create instance.
	
	hit CTRL+X ----> Y -------> Enter to save main.tf file.
	
	Script:
	
	# 1. Define AWS provider
	provider "aws" {
	region = "ap-south-1" # change to your preferred AWS region
	}
	
	# 2. Create a basic EC2 instance
	resource "aws_instance" " ltt_aeh_instance" {

	ami = "ami-0f58b397bc5c1f2e8" # Ubuntu 22.04 LTS (for ap-south-1)
	instance_type = "t3.micro" # Free tier eligible
	
	tags = {
	Name = " ltt_aeh_instance"
	}
	}
	
	Note: If you are using an Organizational AWS account, you must enable EBS encryption in your Terraform script when creating an EC2 instance.
	Add the following configuration inside the aws_instance resource block of your script:
	
	root_block_device {
	volume_size = 8 # in GB
	volume_type = "gp3" # general purpose SSD
	encrypted = true # encryption enabled
	}
	
	 
	
	Please replace your project name and region in the script.

5	Initialize Terraform Command: "terraform init" 
6	Check the Execution Plan Command: “terraform plan”

	
7	Apply Terraform to Create Resources Command: terraform apply -auto-approve 
8	check ltt_aeh_instance resource at AWS cloud console. In the AWS Management Console, navigate to **EC2 → Instances**, then switch to the **region (ap-south-1)** where your instance was created.



9 After creating instance, make sure you have delete it using following command:
terraform destroy -target=<resource_name>

terraform destroy -target=aws_instance.ltt_aeh_instance

```
ubuntu@ip-172-31-23-0:~$ terraform destroy -target=aws_instance.ltt_aeh_instance
aws_instance.ltt_aeh_instance: Refreshing state... [id=i-022e55cda998ffb0c]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  - destroy

Terraform will perform the following actions:

  # aws_instance.ltt_aeh_instance will be destroyed
  - resource "aws_instance" "ltt_aeh_instance" {
      - ami                                  = "ami-0f58b397bc5c1f2e8" -> null
      - arn                                  = "arn:aws:ec2:ap-south-1:633458675990:instance/i-022e55cda998ffb0c" -> null
      - associate_public_ip_address          = true -> null
      - availability_zone                    = "ap-south-1b" -> null
      - disable_api_stop                     = false -> null
      - disable_api_termination              = false -> null
      - ebs_optimized                        = false -> null
      - force_destroy                        = false -> null
      - get_password_data                    = false -> null
      - hibernation                          = false -> null

Enter a value: yes

aws_instance.ltt_aeh_instance: Destroying... [id=i-022e55cda998ffb0c]
aws_instance.ltt_aeh_instance: Still destroying... [id=i-022e55cda998ffb0c, 10s elapsed]
aws_instance.ltt_aeh_instance: Still destroying... [id=i-022e55cda998ffb0c, 20s elapsed]
aws_instance.ltt_aeh_instance: Still destroying... [id=i-022e55cda998ffb0c, 30s elapsed]
aws_instance.ltt_aeh_instance: Still destroying... [id=i-022e55cda998ffb0c, 40s elapsed]
aws_instance.ltt_aeh_instance: Still destroying... [id=i-022e55cda998ffb0c, 50s elapsed]
aws_instance.ltt_aeh_instance: Destruction complete after 51s

Warning: Applied changes may be incomplete

The plan was created with the -target option in effect, so some changes requested in the configuration may have been ignored and the
output values may not be fully updated. Run the following command to verify that no other changes are pending:
    terraform plan

Note that the -target option is not suitable for routine use, and is provided only for exceptional situations such as recovering from
errors or mistakes, or when Terraform specifically suggests to use it as part of an error message.


Destroy complete! Resources: 1 destroyed.
```

Note: Delete resources **by specifying their names only**; otherwise, Terraform will remove **all** the resources it created.

Activity 4: Create AWS S3 Bucket using Terraform

4.1 Prerequisite

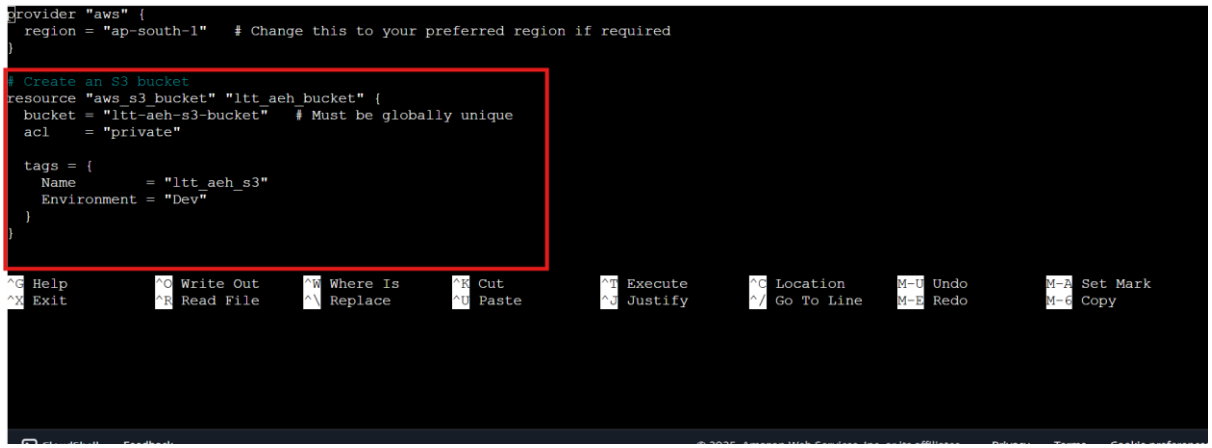
AWS cloud account is ready. **Terraform is authenticated and configured to create resources.**

4.2 Scenario

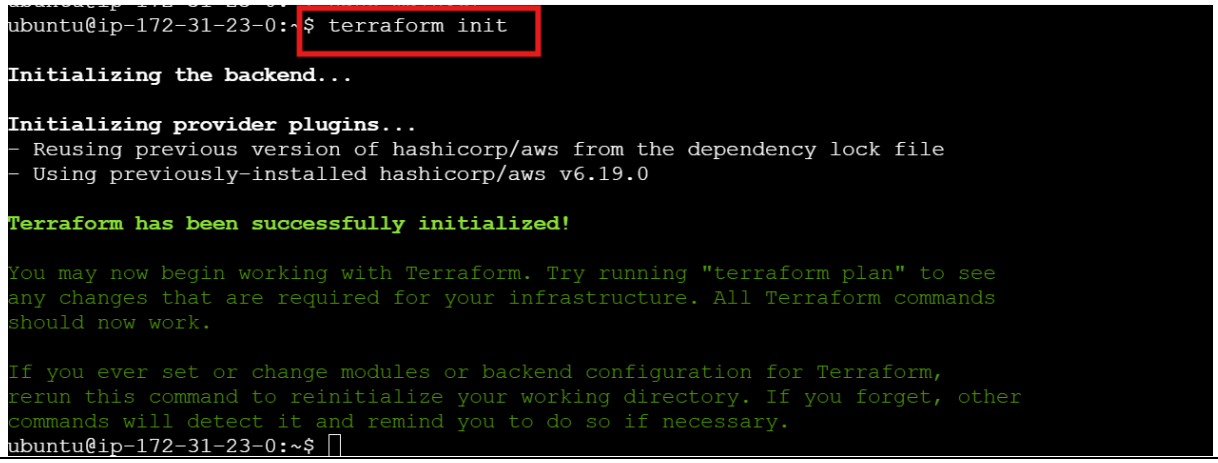
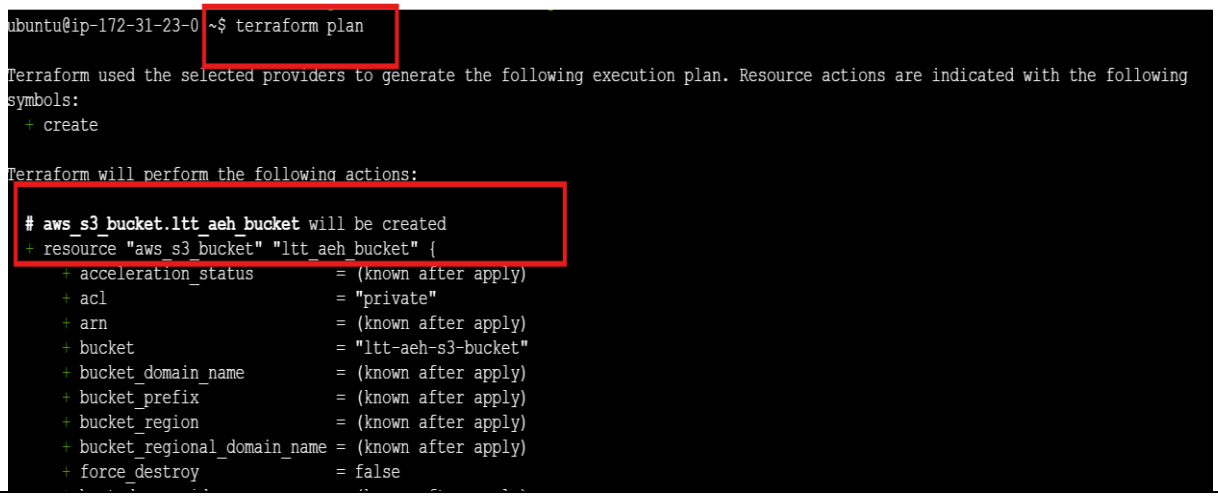
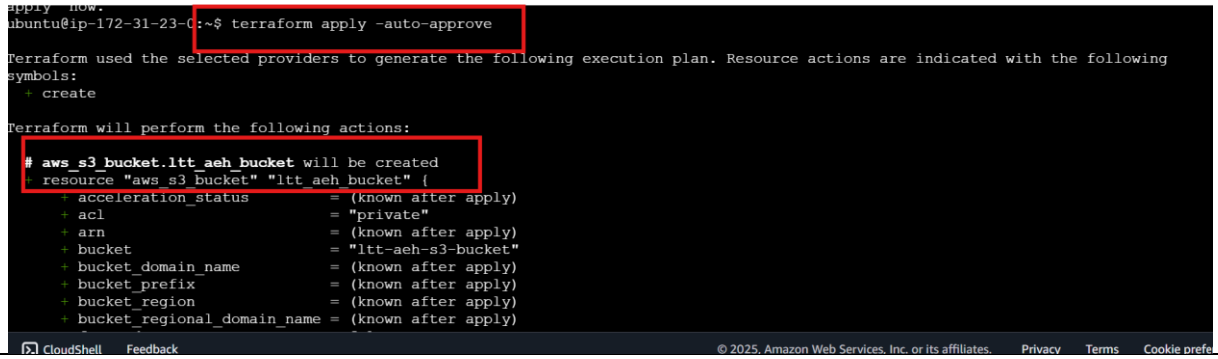
Create AWS cloud S3 bucket resource using terraform script.

4.3 Steps

1	Make sure you have authenticated AWS. Verify using : aws sts get-caller-identity
2	If AWS authentication is not yet configured, run the "**aws configure**" command and provide the required credentials when prompted.

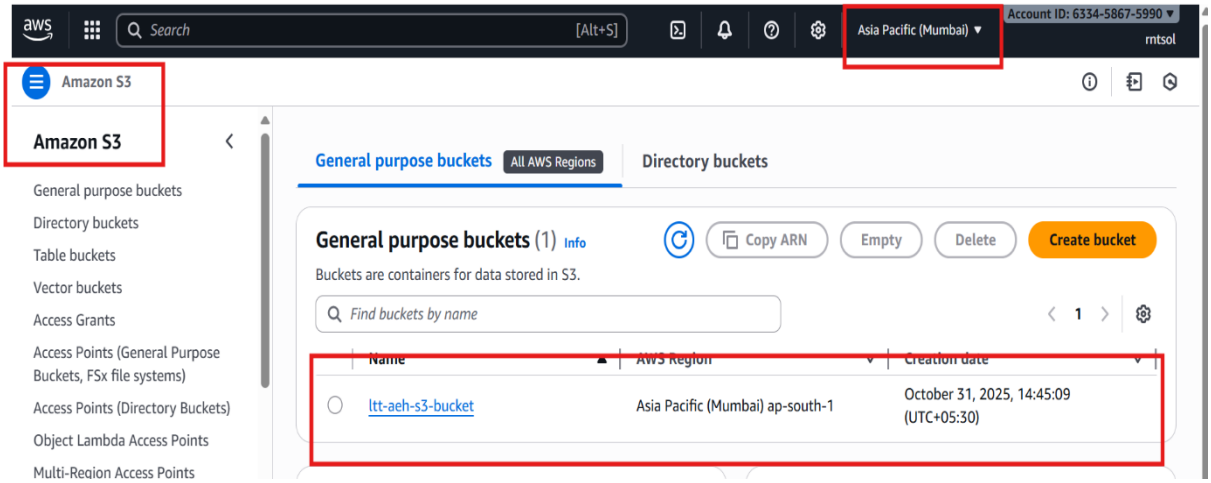
3	create .tf script file to create resources.
	
	execute "nano main.tf"
	
	`ubuntu@ip-172-31-23-0:~$ nano main.tf`
	`ubuntu@ip-172-31-23-0:~$`
	
	paste attached terraform script in this file to create vm
	
	 
	Script:
	provider "aws" {
	region = "ap-south-1" # Change this to your preferred region if required
	}
	
	# Create an S3 bucket
	resource "aws_s3_bucket" "ltt_aeh_bucket" {
	bucket = "ltt-aeh-s3-bucket" # Must be globally unique
	acl = "private"
	
	tags = {
	Name = "ltt_aeh_s3"
	Environment = "Dev"
	}
	}
	hit CTRL+X ----> Y -------> Enter to save main.tf file.

5	Initialize terraform

	Command: terraform init 
6	Check terraform plan. Command: terraform plan 
7	Apply Terraform to Create Resources Command: terraform apply -auto-approve 

```
Warning: Argument is deprecated

  with aws_s3_bucket.ltt_aeh_bucket,
  on main.tf line 8, in resource "aws_s3_bucket" "ltt_aeh_bucket":
   8:   acl    = "private"

acl is deprecated. Use the aws_s3_bucket_acl resource instead.

(and 2 more similar warnings elsewhere)

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

8 In the AWS Management Console, navigate to **Amazon S3**, then switch to the **region (ap-south-1)** where your bucket was created.



9 After creating bucket, make sure you have delete it using following command:
terraform destroy -target=aws_s3_bucket.<bucket_name>

terraform destroy -target=aws_s3_bucket.ltt_aeh_bucket

```
ubuntu@ip-172-31-23-0:~$ terraform destroy -target=aws_s3_bucket.ltt_aeh_bucket
aws_s3_bucket.ltt_aeh_bucket: Refreshing state... [id=ltt-aeh-s3-bucket]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  - destroy

Terraform will perform the following actions:

  # aws_s3_bucket.ltt_aeh_bucket will be destroyed
  - resource "aws_s3_bucket" "ltt_aeh_bucket" {
      - acl                         = "private" -> null
      - arn                         = "arn:aws:s3:::ltt-aeh-s3-bucket" -> null
      - bucket                      = "ltt-aeh-s3-bucket" -> null
      - bucket_domain_name          = "ltt-aeh-s3-bucket.s3.amazonaws.com" -> null
      - bucket_region               = "ap-south-1" -> null
      - bucket_regional_domain_name = "ltt-aeh-s3-bucket.s3.ap-south-1.amazonaws.com" -> null
      - force_destroy               = false -> null
      - hosted_zone_id              = "Z11RGJOFQNVJUP" -> null
      - id                          = "ltt-aeh-s3-bucket" -> null
      - object_lock_enabled         = false -> null
      - region                      = "ap-south-1" -> null
      - request_payer               = "BucketOwner" -> null
      - tags                        = {
aws_s3_bucket.ltt_aeh_bucket: Destroying... [id=ltt-aeh-s3-bucket]
aws_s3_bucket.ltt_aeh_bucket: Destruction complete after 1s

Warning: Applied changes may be incomplete

The plan was created with the -target option in effect, so some changes requested in the configuration may have been ignored and the
output values may not be fully updated. Run the following command to verify that no other changes are pending:
    terraform plan

Note that the -target option is not suitable for routine use, and is provided only for exceptional situations such as recovering from
errors or mistakes, or when Terraform specifically suggests to use it as part of an error message.


Destroy complete! Resources: 1 destroyed.
```


Activity 5: Creating AWS RDS using Terraform

5.1 Prerequisite

AWS cloud account is ready.

Terraform is authenticated and configured to create resources.

5.2 Scenario

Create a simple and minimal Terraform script to launch an AWS RDS (MySQL) instance.

5.3 Steps

1	Make sure you have authenticated AWS. Verify using : aws sts get-caller-identity
2	If AWS authentication is not yet configured, run the "**aws configure**" command and provide the required credentials when prompted.
3	Create new terraform file create .tf file to create resources. execute "nano main.tf"

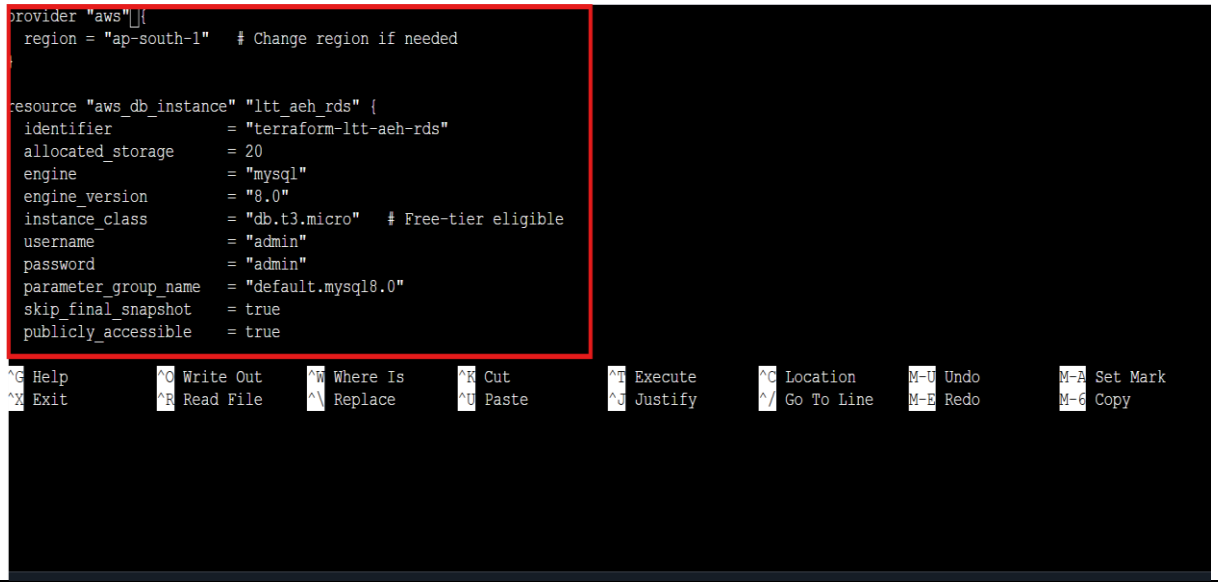
4	copy and paste following script in main.tf file
	
	provider "aws" {
	region = "ap-south-1" # Change region if needed
	}
	
	resource "aws_db_instance" "ltt_aeh_rds" {
	identifier = "terraform-ltt-aeh-rds"
	allocated_storage = 20
	engine = "mysql"
	engine_version = "8.0"
	instance_class = "db.t3.micro" # Free-tier eligible
	username = "admin"
	password = "Admin#12345"
	parameter_group_name = "default.mysql8.0"
	skip_final_snapshot = true
	publicly_accessible = true

```
  tags = {
    Name = "terraform-ltt-aeh-rds"
  }
}
```

Note: If you are using an Organizational AWS account, you must enable EBS encryption in your Terraform script when creating an RDS instance.
Add the following configuration inside the RDS "resource" block of your script:

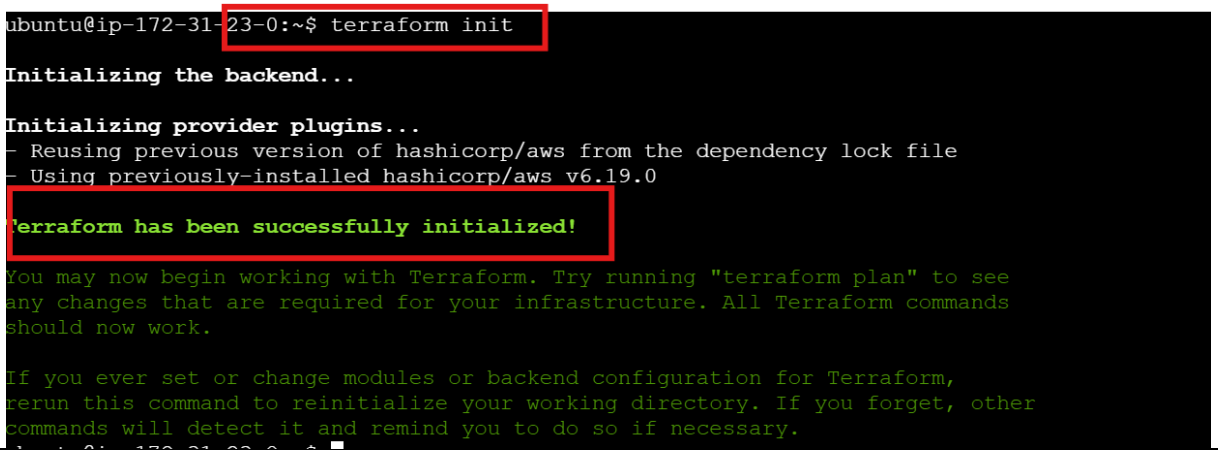
storage_encrypted = true

hit CTRL+X ----> Y -------> Enter to save file.



5 Initialize Terraform

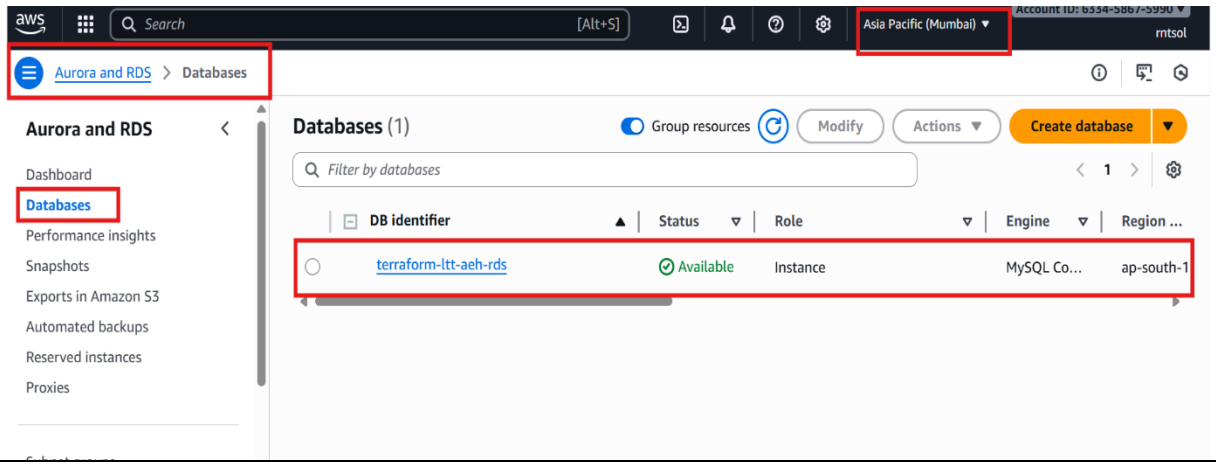
terraform init



6 Check execution plan

terraform plan

	ubuntu@ip-172-31-23-0:~$ terraform plan Terraform used the selected providers to generate the following execution plan. Resource actions are indica symbols: + create Terraform will perform the following actions: # aws_db_instance.ltt_aeh_rds will be created + resource "aws_db_instance" "ltt_aeh_rds" { + address = (known after apply) + allocated_storage = 20 + apply_immediately = false + arn = (known after apply) + auto_minor_version_upgrade = true + availability_zone = (known after apply) + backup_retention_period = (known after apply) + backup_target = (known after apply) + backup_window = (known after apply) + ca_cert_identifier = (known after apply) + character_set_name = (known after apply) + copy_tags_to_snapshot = false + database_insights_mode = (known after apply)
7	Apply & Create the Cloud SQL Instance terraform apply -auto-approve ubuntu@ip-172-31-23-0:~$ terraform apply -auto-approve Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols: + create Terraform will perform the following actions: # aws_db_instance.ltt_aeh_rds will be created + resource "aws_db_instance" "ltt_aeh_rds" { + address = (known after apply) + allocated_storage = 20 + apply_immediately = false + arn = (known after apply) + auto_minor_version_upgrade = true + availability_zone = (known after apply) + backup_retention_period = (known after apply) + backup_target = (known after apply) + backup_window = (known after apply) + ca_cert_identifier = (known after apply) + character_set_name = (known after apply) + copy_tags_to_snapshot = false CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie aws_db_instance.ltt_aeh_rds: Still creating... [2m40s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [2m50s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m0s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m10s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m20s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m30s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m40s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m50s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m0s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m10s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m20s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m30s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m40s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m50s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [5m0s elapsed] aws_db_instance.ltt_aeh_rds: Creation complete after 5m10s [id=db-DHATQ54NNQ3JITDBM3FHLPZSQI] Apply complete! Resources: 1 added, 0 changed, 0 destroyed. ubuntu@ip-172-31-23-0:~$
8	RDS creation isn't instant — even a small instance like db.t3.micro takes 5-15mins. You can go to: AWS Console → RDS → Databases-> instance
9	Verify the RDS Instance Go to AWS Console → RDS → Databases-> instance You should see your new AWS RDS instance:

	
10	Delete RDS instance: terraform destroy -target=aws_db_instance.ltt_aeh_rds

Activity 6: Create AWS IAM Role using Terraform

6.1 Prerequisite

AWS cloud account is ready.

Terraform is authenticated and configured to create resources.

6.2 Scenario

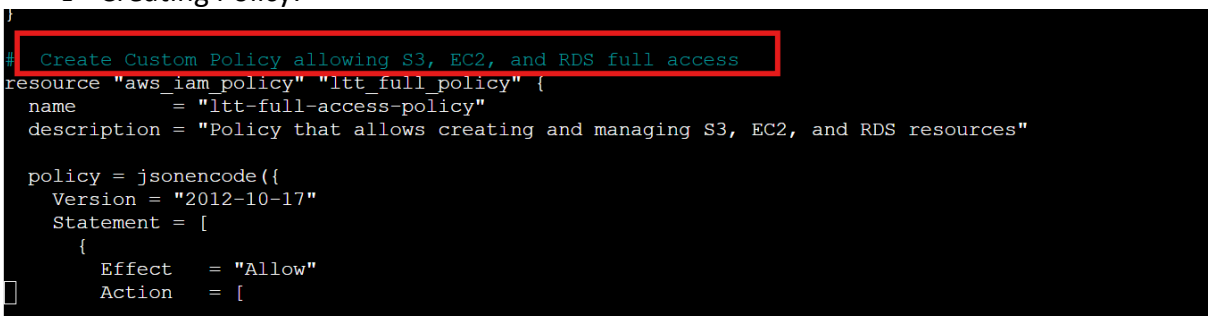
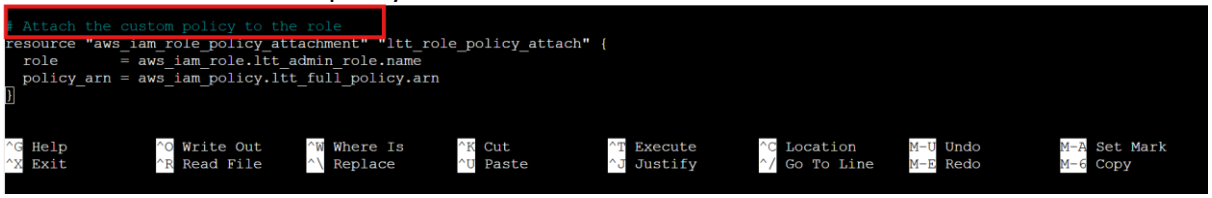
To Create AWS IAM Role using terraform.

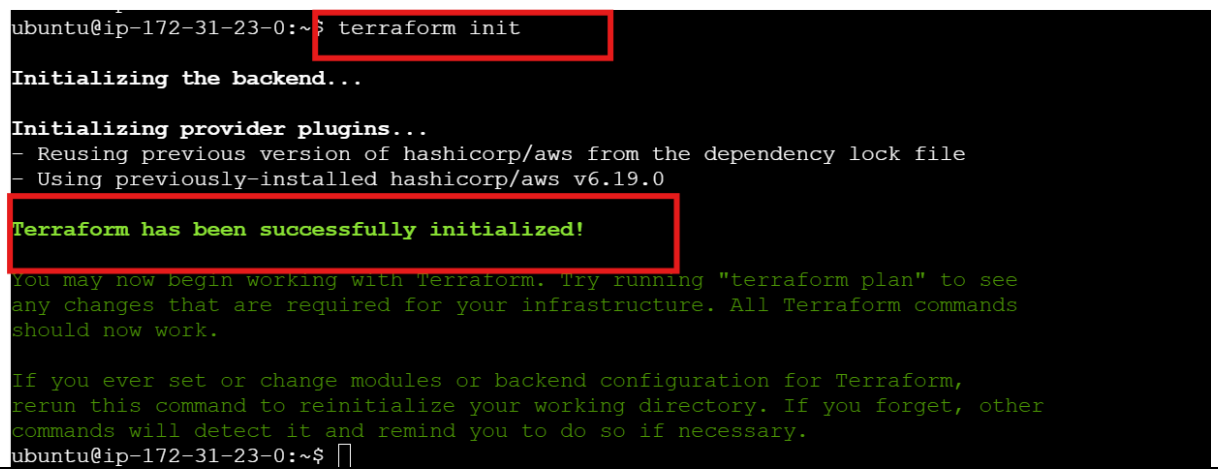
6.3 Steps

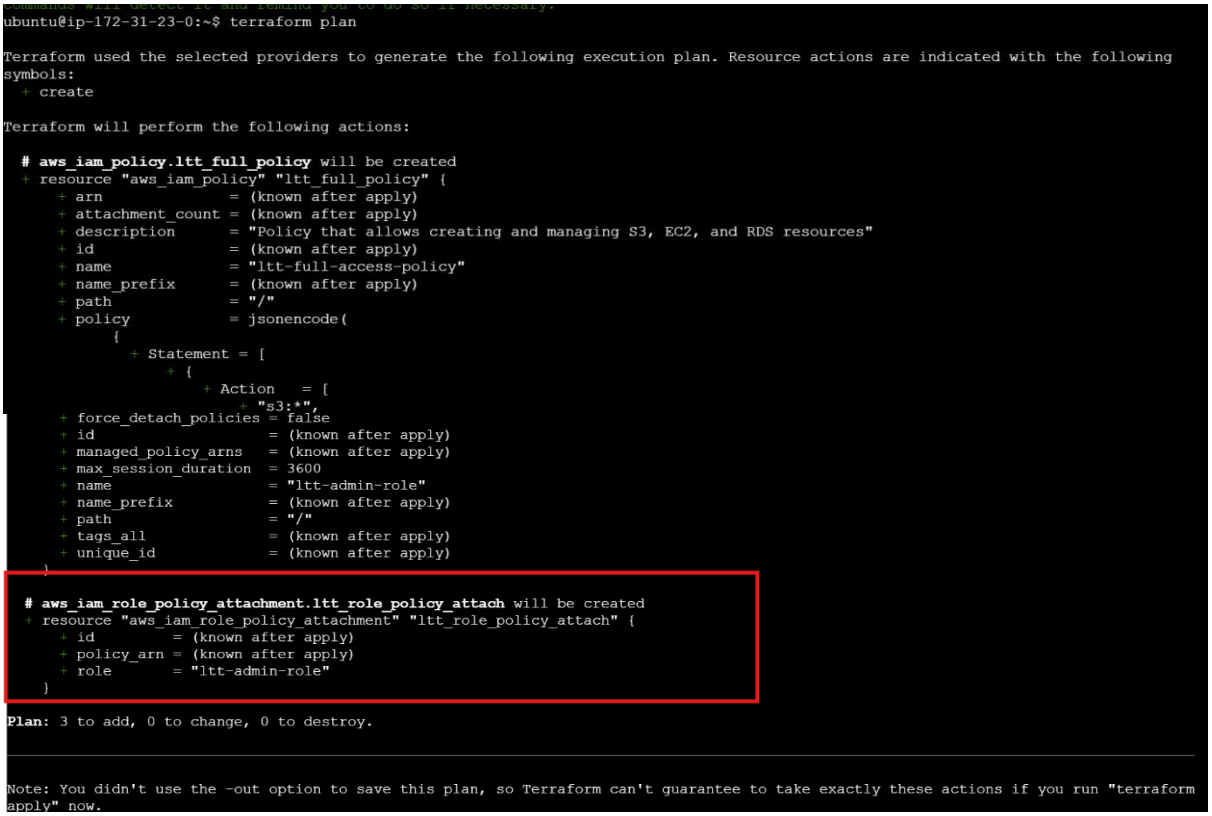
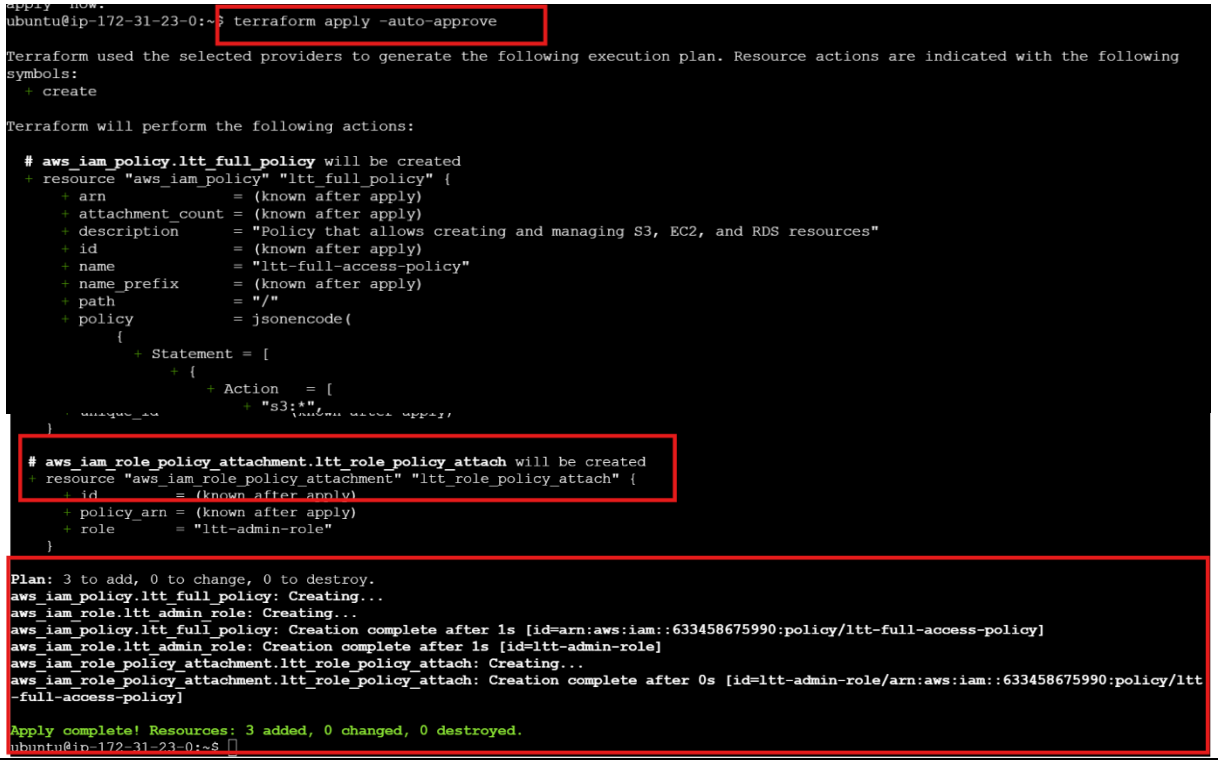
1	Make sure you have authenticated AWS. Verify using : aws sts get-caller-identity
2	If AWS authentication is not yet configured, run the "**aws configure**" command and provide the required credentials when prompted.
3	Create new terraform file create .tf file to create resources. execute "nano main.tf"

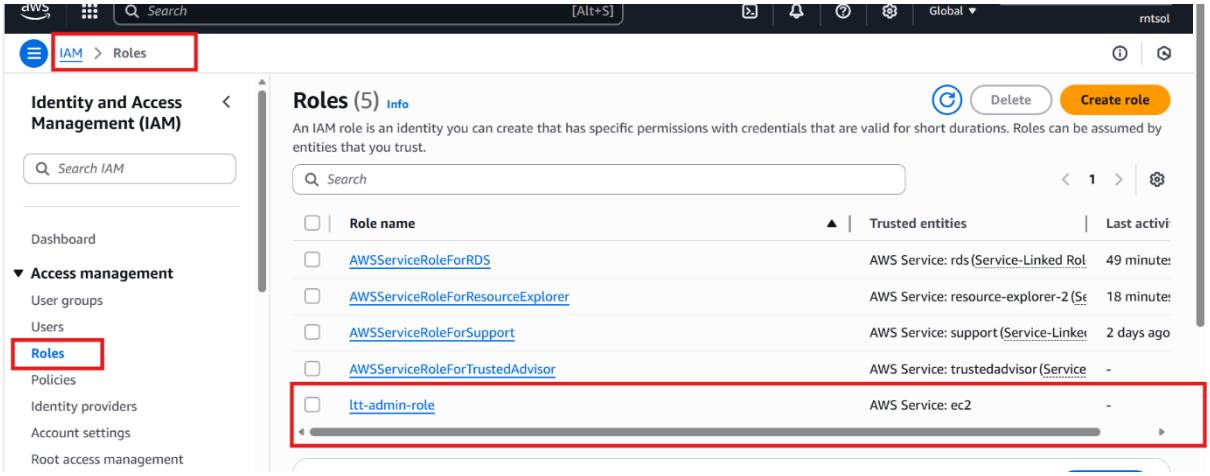
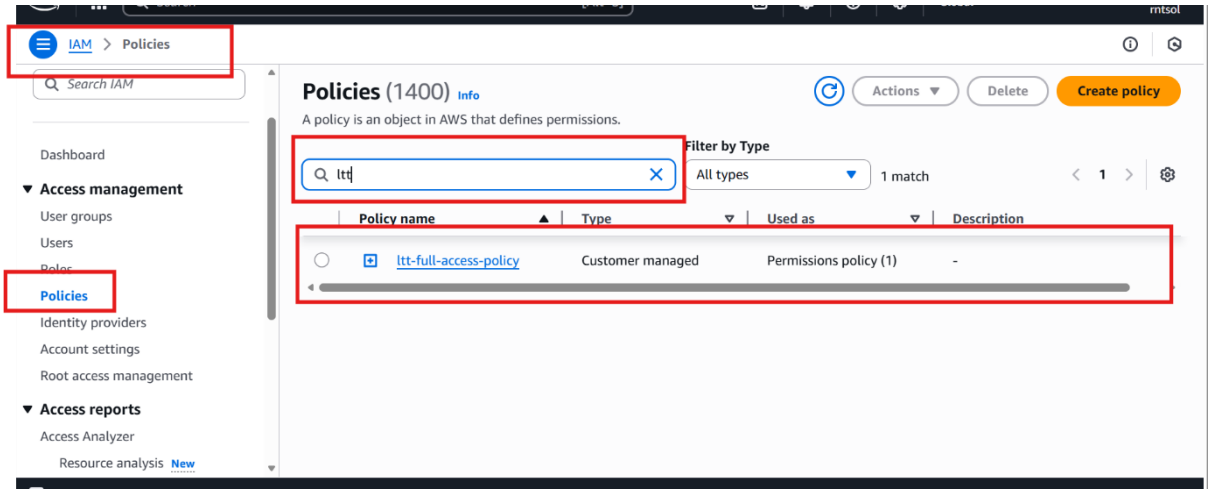
4 Copy paste given script in main.tf file

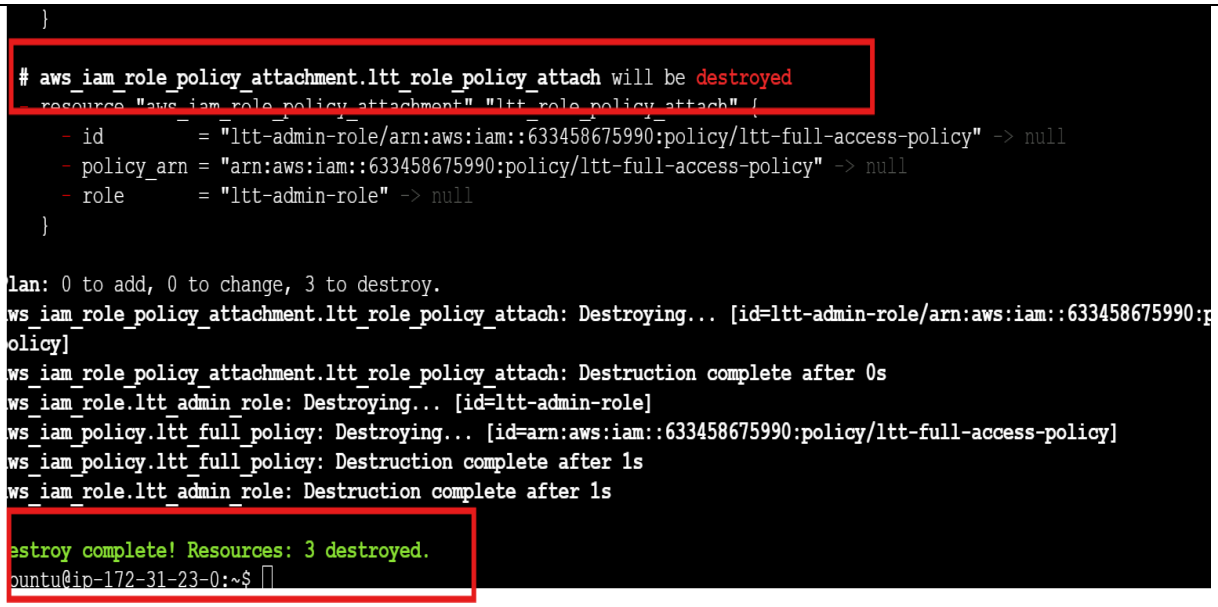
```
# AWS provider
provider "aws" {
  region = "ap-south-1"
}

# Create IAM Role
resource "aws_iam_role" "ltt_admin_role" {
  name = "ltt-admin-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "ec2.amazonaws.com"  # allows EC2 instances to assume this role
        }
      }
    ]
  })
}

#  Create Custom Policy allowing S3, EC2, and RDS full access
resource "aws_iam_policy" "ltt_full_policy" {
  name        = "ltt-full-access-policy"
  description = "Policy that allows creating and managing S3, EC2, and RDS resources"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect   = "Allow"
        Action   = [
          "s3:*",
          "ec2:*",
          "rds:*",
          "iam:PassRole"
        ]
        Resource = "*"
      }
    ]
  })
}
```

	# Attach the custom policy to the role
	resource "aws_iam_role_policy_attachment" "ltt_role_policy_attach" {
	role = aws_iam_role.ltt_admin_role.name
	policy_arn = aws_iam_policy.ltt_full_policy.arn
	}
	
	➔ Creating IAM Role:
	 
	➔ Creating Policy:
	 
	➔ Attached custom policy to the role:
	 
	
	hit CTRL+X ----> Y -------> Enter to save script

5	Initialize terraform terraform init 
6	

	Check execution plan terraform plan 
7	Apply & Create the Role: terraform apply -auto-approve 

7	Verify custom role and policy created using terraform script Go to AWS Console → IAM Dashboard → Roles:  Go to AWS Console → IAM Dashboard → Policies:  You should see the newly created role "**my Custom Role and Policy**".
8	Destroy the Role: To delete the custom role, execute following command: terraform destroy -auto-approve

	
10	*Note: We can add more permissions or assign roles dynamically using Terraform script*